

Data Types

Overview

- "Data encoded to a program is stored in memory and can have and be in several types—for example, using a person's basic information such as name, age, and address."
- "Data Types describe possible operations on the data and the storage method."
- "The data types in Java are classified into two (2): primitive or standard data types and abstract or derived data types."

Primitive Data Types (Agarwal & Bansal, 2023)

- "Primitive Data Types are built into the Java language."
- "These are data that are not objects and have no methods."
- "The Java compiler has detailed instructions on the valid operations this data type supports."

Primitive Types Summary

- **byte**: "Byte Length Integer", 8 bit, Default: 0, Range: -128 to +127 signed.
- **short**: "Short Integer", 16 bit, Default: 0, Range: -32,768 to +32,767.
- **int**: "Integer (number)", 32 bit, Default: 0, Range: -2,147,483,648 to 2,147,484.
- **long**: "Long Integer", 64 bit, Default: 0L, Range: -2^{63} to $2^{63}-1$.
- **float**: "Single Precision Floating Point", 32 bit, Default: 0.0f, Range: $\pm 1.4 \times 10^{-45}$ to $\pm 3.4 \times 10^{38}$.
- **double**: "Double-Precision Floating Point", 64 bit, Default: 0.0d, Range: $\pm 4.9 \times 10^{-324}$ to $\pm 1.8 \times 10^{308}$.
- **char**: "A simple character", 8 bit, Default: `\u0000`, Range: 0 to 65,535.
- **Boolean**: "A Boolean (true or false)", 1 bit, Default: false.

Integer Types

- Java defines Byte, Short, Int, and Long as Integer data types.
- These four types are signed (positive and negative values) because Java does not support unsigned integers.
- **Byte**: "The smallest integer type that represents a small number." Syntax: `byte b;`
- **Short**: An integer value between -32768 and 32767. Hexadecimal numbers can also be initialized using `0x` (e.g., `0x4A8` for 3001).
- **Int**: "Can handle larger numbers and is considered the best choice when an integer is needed because when Byte and Short values are used in an expression, they are promoted to Int automatically."
- **Long**: Holds a very large number, with a capital letter 'L' placed right after the value.

Floating Point Types

- Floating points are also called "real numbers" and are used when evaluating expressions needing fractional accuracy.
- **Float**: "Described as a single precision floating point." Uses a small letter 'f' to the right of the value when initializing.

- **Double:** "Double precision can be faster than single precision on modern processors that have been optimized for high-speed mathematical calculations." It handles larger numbers than Float.

Character Types

- Divided into (a) Byte for a character (ASCII) and (b) Char (Unicode).
- **ASCII / Byte:** A character recognized by its ASCII number, allowing Byte data types to hold a single quoted character.
- **Unicode / Char:** Created to support non-traditional and non-Latin-based languages, initialized using single quotes.

Boolean

- "This data type is used for logical values and can only hold true or false values." It is a type returned by all relational operators such as `=`, `>`, and `<`.

Abstract Data Types

- "These are based on primitive data types but with more functionality."
- **String:** An abstract data type that can store letters, digits, and special characters, and provides methods for linking, searching, and extracting portions of strings.

Constants and Variables (Agarwal & Bansal, 2023)

- "The values of variables can be changed in a program, but the value of constants, cannot."
- **Variable Naming Conventions:**
 - Must be meaningful, short, without embedded spaces or symbols (except underscores `_`).
 - Must be unique and begin with a letter, a dollar sign, or an underscore.
 - Keywords cannot be used for variable names (e.g., `switch`).
 - Java convention: Nouns starting with a lowercase letter, with each subsequent word beginning with an uppercase letter.
- **Declaring and Initializing:** The basic format is `Type identifier [= value] [, identifier] [= value];`. Java returns an error if a variable declared in a method is not initialized.

Literals

- **Boolean literals:** `true` or `false`.
- **Integer literals:** Can be octal (prefixed with `0`), hexadecimal (prefixed with `0x`), or binary (starts with `0B` or `0b`), and can use underscores (`_`) for readability.
- **Floating-point literals:** Numbers with a decimal fraction requiring an `F` (lower or uppercase) suffix to avoid compiler warnings or errors about loss of precision.
- **Character literals:** Enclosed in single quotes (`'`), including Unicode values or escape characters using a backslash (`\`).
- **String literals:** Enclosed in double quotes (`"`, `"`).